BLA 2: Software Processes in Action

Transcript / Technical Report
Software Engineering I | Dr. Victor Govindaswamy
Austin Koberstein | Fall 2026

Purpose and Course Concepts

This report follows the Lab 02 requirements and Lecture 2 topics: software process models, process activities, coping with change, and process improvement. It covers Waterfall, incremental development, integration/configuration, plan-driven and agile development, the four fundamental activities, two real-world applications, a comparison, and a reflection on process choice.

Presentation Transcript

Slide 1: Software Processes in Action

Hello. Within the framework of the BLA 2 presentation, I will discuss software processes, including process models, the four basic activities, how teams deal with change, and examine two cases: the GitHub.com community and the Netflix streaming service. Moreover, I will compare these two objects and explain why different development processes are often used within one company.

Slide 2: What Is a Software Process?

A software process can be defined as a set of activities required for developing and operating a software system. The lecture provides four basic activities which can be seen in the image below. Firstly, specification determines system requirements. Then, design and implementation activities mean defining the design and developing the system. The validation task is to check if the system meets the customer’s requirements, and finally, the evolution deals with changing requirements and modifying the system.

Slide 3: Why Use an Organized Process?

The software development process is more complicated than programming a system. It is also necessary to consider group interactions during requirement analysis, staffing, design, testing, documenting, releasing, and maintaining. A software process model is an abstraction of a real process that can be viewed from different perspectives, and it is essential to choose a specific model when studying a process. The model allows the group to focus on significant aspects, and the lecture highlights that in practice, prototyping is used extensively. However, the models are only a simplified description of reality, not a detailed and prescribed set of instructions. Moreover, many projects use a mixture of the approaches described, and therefore, these models can serve only as a guideline.

Slide 4: Major Software Process Approaches

The major approaches in this assignment are Waterfall, incremental development, integration and configuration, plan-driven development, and agile development. Waterfall separates work into distinct phases. Incremental development overlaps specification, development, and validation and delivers software in pieces. Integration and configuration starts from reusable software or services and adapts them to the project. Plan-driven development plans activities and progress in advance. Agile development plans in smaller cycles and makes change easier to absorb. A project may use more than one of these approaches.

Slide 5: Waterfall Model

Requirements definition, system and software design, implementation and unit testing, integration and system testing, and operation and maintenance are all phases in the Waterfall model. Due to the phases, outputs, schedule, and documentation being defined prior to any further development taking place it is known as plan driven. Due to the fact that each phase will create the necessary information for the next it is imperative that documentation is kept up to date. Should a change be made to a requirement after it has been coded the team may need to update requirements, design, implementation, test cases, and documentation. One of the main downsides to the Waterfall method is this potential for rework.

Slide 6: Waterfall: Strengths and Limits

Waterfall has some significant advantages. The phases and gates are easily visible, documentation is prioritized, and managers can control progress based on a specific plan. Waterfall is appropriate for a project with well-defined requirements. The main disadvantages of the Waterfall model are related to the frequent changes of requirements. The users can get results only at the end of the project. Therefore, if new requirements appear, problems may arise that will require a lot of time to be resolved. According to the lecture, large-scale, multi-site, and critical embedded systems are examples of projects for which plan-driven approaches are considered appropriate.

Slide 7: Incremental Development

Incremental development refers to delivering the system as a series of increments that form the end product. This strategy is characterized by specification, development, and validation of each increment. The team specifies and prioritizes requirements, develops and tests them, and then deploys it. In this case, an organization gets early feedback from customers on the delivered requirements, thus enabling the team to make decisions about future development increments. Incremental development makes changes easier to plan and adopt since at a time less work is committed to development. However, there are some challenges associated with incremental development. There is a need to design and implement shared system facilities, and the system architecture should be designed in a way that it accommodates future increments while being easy to maintain.

Slide 8: Integration and Configuration

Integration and configuration are centered around reuse. Following the collection of requirements, the team looks for pre-existing software, services, frameworks, libraries, or configurable systems, assesses them, adjusts the requirements as needed, configures or modifies the chosen components, and integrates them into the finished system. Because the

team does not have to start from scratch when implementing all required features, shorter development times are a significant advantage. The disadvantage is that since the existing components may not have all the features required, the project may have to accept the limitations and dependencies of the selected components.

Slide 9: Software Process Activities and Change

Throughout the lifecycle of a software product, the four process activities are carried out. Its specifications outline its limitations and requirements. The product is created through design and implementation. Validation verifies that it satisfies user requirements and specifications. After delivery, Evolution makes changes to the system. Because business, technology, users, and the environment all have changing needs, the process should be adaptable. Through prototyping, requirements or design choices that may not be evident at first can be better understood. While the project team gains knowledge from the experience, incremental delivery enables the user to acquire some helpful capabilities early on.

Slide 10: Real-World Application 1: GitHub.com

My first real-life example is GitHub.com, which is the product of GitHub. GitHub.com is a source-code repository, pull-requests, code review, automation, and collaboration platform for developers and organizations. GitHub's engineering blog posts about the development process, which involve frequent development cycles, continuous integration, pull request reviews, merge queues, staged deployments, and internal developer feedback. Considering the characteristics, I can say that it is more likely to be agile and incremental rather than Waterfall, as there are multiple stages, including DevOps automation.

Slide 11: GitHub.com Development Process

A simplified GitHub.com process starts from a developer or product need, where engineers ship a change on a branch and open a pull request. The automated continuous integration builds it and runs tests, and there's a review before it gets added to the merge queue. GitHub does report that merge queue groups together candidate pull requests, runs builds and tests, detects conflicts, and prevents the main branch from moving to a failing commit. They also report that they run large-scale CI using GitHub Actions. Once validated, it goes through deployment stages and production monitoring, after which feedback and operations data inform the next change.

Slide 12: Real-World Application 2: Netflix Streaming

My second application is the Netflix streaming system, provided by Netflix. This application is used by its members on multiple devices, enabling them to discover and view videos online. The company's engineering team has published many articles about its heavy use of personalization, data-driven decision making, distributed systems, and automated testing. The Simone test platform was designed and implemented to replicate the environment of all Netflix services and test how different devices react to it. This evidence supports the opinion that Netflix uses a progressive and experimental culture of developing products within an extremely integrated and complex environment.

Slide 13: Netflix Development Process

A streamlined version of the netflix business process starts with a member need, product goal, or observation. Teams scope out a requirement or experiment, design the change to the application, service, or recommendation model, implement the change, and test it. Netflix's Simone system allows for creating controlled scenarios to simulate devices and services. The personalization team bases its experiments on data about the members' preferences on the homepage. Once an experiment is implemented and released, the results are analyzed through telemetry and member response, informing the next round of experiments. In short, it is an iterative process with feedback loops and not a linear endeavor that follows a predefined set of phases.

Slide 14: GitHub.com vs. Netflix

GitHub.com and Netflix have much in common in terms of their practices. Both use automation, validation, feedback, and continuous evolution. However, the emphasis is different for these two companies since their products are different. GitHub's public engineering evidence is concentrated in implementing safe merging of multiple pull requests, running CI, and deploying changes. For Netflix, the attention is drawn to distributed-service behavior, device testing, personalization, and experimentation. Thus, the two companies used the same process model in different ways, according to their needs.

Slide 15: Comparison Table

The comparison table of the two companies shows some apparent dissimilarities and similarities. Firstly, while GitHub is primarily focused on developers and organizations, Netflix is geared toward streaming members and device partners. Moreover, the former uses pull requests, automated CI, merge queues, and staged deployment, while the latter utilizes services, client applications, personalization, simulation testing, and experiments. Notably, both companies are agile and incremental. The main challenge of GitHub is the need to maintain a high speed of development and effectively incorporate numerous contributions. In turn, the central problem of Netflix is the delivery of a consistent experience across multiple platforms and services and constant enhancement of the product.

Slide 16: Why Different Processes Work

The peculiar limitations that each business encounters may explain the differences in the procedures that two successful companies implement. Risks, dependencies, quality, resources, change control, customer expectations, regulations, costs, and time-to-market are some of the aspects of project management. Given the different services that GitHub and Netflix offer, the requirements for each process are distinct. If Netflix highlights the importance of streaming's reliability, the distributed system, devices, and personalization, GitHub focuses on the ability to host a constant flow of code changes. The key principles of project management applied to large-scale projects include combinations of plan-driven approaches and controlled changes through iterative and agile methods.

Slide 17: What I Learned

What is the main takeaway from the assignment? Please briefly describe your thoughts.

The main takeaway is that there is no silver bullet development process that fits all. Waterfall is appropriate for projects with rigid requirements and where documentation is paramount. Processes like incremental development allow for value delivery early and the option to change course. Pairing the processes with the use of automated testing and feedback loops allows for reduced amount of code that needs to be developed, deployed, and manually tested. All the described processes fit best in certain scenarios defined by the risk, users, features, and speed.

Slide 18: Sources

These are the sources that were used for the actual application research. The GitHub link and the article from Netflix show that the claims about testing, deployment, personalization, and development are all things that have happened publicly. Thank you for watching.

Detailed Comparison Table

Category	GitHub.com	Netflix Streaming
Company/Organization	GitHub	Netflix
Software/Application	GitHub.com	Netflix streaming application and supporting services
Primary Users	Developers and organizations	Streaming members and device partners
Development Approach	Agile/incremental with DevOps automation	Incremental/experimental with integration-heavy distributed services
Requirements Process	Product needs, developer needs, operational pain points	Member needs, product goals, behavior, device/service requirements
Development	Branches, pull requests, platform/services	Client apps, distributed services, personalization algorithms/models
Testing/Validation	Automated CI, branch protection, merge queue checks, staged rollout	Simulation, device certification, service testing, experiments
Deployment	Automated pipeline and staged deployment	Distributed service and client releases, controlled evaluation
Maintenance/Updates	Frequent production changes and platform evolution	Continuous service, device, and personalization updates
Customer Feedback	Customer needs plus internal developer experience feedback	Member behavior, product metrics, experiment results
Waterfall Characteristics	Some planning, documentation, controlled rollout, but not strict sequential phases	Some structured validation and device certification, but not strict sequential phases
Agile/Incremental Characteristics	Frequent small changes, automated feedback, continuous delivery	Repeated experiments, feedback loops, ongoing evolution
Major Advantage	High shipping throughput with automated safety checks	Fast learning and adaptation across a large member base
Major Challenge	Safely integrating many concurrent changes at scale	Reliable behavior across many devices and distributed services

Original Process Diagrams

Waterfall: Requirements Definition → System and Software Design → Implementation and Unit Testing → Integration and System Testing → Operation and Maintenance

Incremental: Initial Requirements → Increment 1 → Feedback → Increment 2 → Feedback → Increment 3 → Continued Evolution

GitHub.com: Need/Issue → Branch + Pull Request → Automated CI → Review → Merge Queue → Staged Deployment → Monitoring/Feedback → Next Change

Netflix: Member Need/Data → Requirement or Experiment → Design/Model/Service Change → Implementation → Simulation/Testing → Evaluation → Deployment → Telemetry/Feedback → Next Iteration

References

GitHub. How GitHub uses merge queue to ship hundreds of changes every day. https://github.blog/engineering/engineering-principles/how-github-uses-merge-queue-to-ship-hundreds-of-changes-every-day/

GitHub. How GitHub uses GitHub Actions and Actions larger runners to build and test GitHub.com. https://github.blog/engineering/infrastructure/how-github-uses-github-actions-and-actions-larger-runners-to-build-and-test-github-com/

Netflix. Simone - A Distributed Simulation Service. https://netflixtechblog.com/https-medium-com-netflix-techblog-simone-a-distributed-simulation-service-b2c85131ca1b

Netflix. Learning a Personalized Homepage. https://netflixtechblog.com/learning-a-personalized-homepage-aa8ec670359a

Course materials: Lab 02, Lecture 2 Software Processes Part 1, and Lecture 2 Software Processes Part 2, Dr. Victor Govindaswamy.